

PROGRAMMING II

JAVA BASIC SYNTAX

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

JAVA SIMPLE CONSTRUCTS



VARIABLES

A **variable** is a memory location with an associated type (primitive or reference)

A variable **always** contains data compatible with its type

The value of a variable can change because of assignments or increments

All types have a default value, used at variable initialization

- ▶ 0 for **int**, **null** for references, **false** for **boolean**, ...
- ▶ Automatic initialization is done on class fields only (pay attention)

DEFAULT VALUES

- ✕ No native support for unsigned numbers

Type	Value Range	Default Value
byte	Integers from -2^7 to $2^7 - 1$	0
short	Integers from -2^{15} to $2^{15} - 1$	0
int	Integers from -2^{31} to $2^{31} - 1$	0
long	Integers from -2^{63} to $2^{63} - 1$	0L
float	Rationals with 6 to 7 decimals	0.0f
double	Rationals with 15 to 16 decimals	0.0
boolean	Truth values	false
char	Single ASCII values	'\u0000'
String (any object)	Non-primitive data	null

VARIABLE DECLARATIONS

Variables must be **declared** before their use

```
int i;   or   int i = 5;
```

Declarations may appear everywhere in the code

- ▶ Inside or outside class methods, inside branches or loop body, ...

Declaration and initialization may be done at different times

- ▶ Objects require explicit initialization before use, to avoid `NullPointerException` errors

CONSTANTS

A variable can be declared as a **constant** by using the **final static** keywords

```
final static int I = 5;
```

A constant must be initialized, and its value cannot be changed subsequently

```
int j;
final static int I; // compilation error
final static int I = 6;

j = 5;
I = 3; // compilation error
```

1
2
3
4
5
6

IDENTIFIERS

An **identifier** is a name used to refer variables, methods, classes, constants, ...

Technically, an unbounded sequence of characters in [a-z], [A-Z], [0-9], plus _ and \$

Naming conventions:

- ▶ Use camel casing for compound names w/o underscores
- ▶ Class names must be capitalized, e.g., AerialDrone
- ▶ Methods, objects, fields, method variables must start in lowercase, e.g., getCoordinates(), myCar
- ▶ Constants should be all uppercase w/ underscores, e.g., PI_CONST = 3.14

IDENTIFIERS

A **keyword** is a reserved sequence of characters, not allowed as identifier

abstract	default	if	private	throw
boolean	do	implements	protected	throws
break	double	import	public	transient
byte	else	instanceof	return	try
case	extends	int	short	void
catch	final	interface	static	volatile
char	finally	long	super	while
class	float	native	switch	
const	for	new	synchronized	
continue	goto	package	this	

SPECIAL CHARACTERS

Encoding graphical characters to use in output formatting

```
\b  /* \u0008: backspace BS */
\t  /* \u0009: horizontal tab HT */
\n  /* \u000a: linefeed LF */
\f  /* \u000c: form feed FF */
\r  /* \u000d: carriage return CR */
\"  /* \u0022: double quote */
\'  /* \u0027: single quote */
\\  /* \u005c: backslash */
```

COMMENTS

Single-line comments `// ...`

- ▶ What is after `//` is ignored

Multi-lines comments `/* ... */`

- ▶ What is enclosed between `/*` and `*/` is ignored

Documentation comments `/** ... */`

- ▶ What is enclosed between `/**` and `*/` is ignored
- ▶ But it can be processed by javadoc to produce documentation

EXPRESSIONS

An **expression** is a term that is eventually evaluated to a value

$$\text{eval}(4) = 4 \quad \text{or} \quad \text{eval}(x + 1) = \text{eval}(x) + \text{eval}(1) = 3 + 1 = 4$$

Thus, expressions have an associated type

- ▶ 4 has type **int**
- ▶ Given **int** x, then x has type **int**
- ▶ Given **int** x, then x + 1.0 has type **double**

// safe implicit cast

EXPRESSIONS (CONT'D)

Expressions may have **side effects**

```
int i = 1; int j = (i=2) * i;
```

What is the value of `i` and `j` after execution?

- ▶ `i` will take 2 while `j` will take 4
- ▶ Expressions are evaluated from left to right

Given `int t = 3`, which is the evaluation of `t + (t=2)`?

$$\text{eval}(t + (t=2)) = \text{eval}(t) + \text{eval}(t=2) = 3 + 2 = 5$$

NOMENCLATURE

Depending on their type, expressions are called

- ▶ Boolean expressions, if they evaluate to **boolean** *// or conditions*
- ▶ Arithmetic expressions, if they evaluate to any primitive type except **boolean**
- ▶ Reference expressions, if they evaluate to any object

We will use the following meta-syntax to refer expressions

<i>expr</i>	Generic expression
<i>Bexpr</i>	Boolean expression
<i>Aexpr</i>	Arithmetic expression
<i>Rexpr</i>	Reference expression

OPERATORS

Used when building expressions to carry out calculations from operands

- ▶ Unary $\ominus Aexpr$ *// e.g., negation*
- ▶ Binary $expr_1 \otimes expr_2$ *// e.g., multiplication*
- ▶ Ternary $Bexpr ? expr_1 : expr_2$ *// inline conditional*

For instance, the expression $x < 0 ? -1 : 1$

- ▶ Evaluates to -1 when x takes -5
- ▶ Evaluates to 1 when x takes 8
- ▶ $expr_1$ and $expr_2$ can have different type

BINARY ARITHMETIC OPERATORS

Subtraction $Aexpr_1 - Aexpr_2$

Division $Aexpr_1 / Aexpr_2$

Addition $Aexpr_1 + Aexpr_2$

Multiplication $Aexpr_1 * Aexpr_2$

Modulo $Aexpr_1 \% Aexpr_2$

Concatenation $Rexpr_1 + Rexpr_2$

Operators **overloading**

- ▶ `int + int` will yield an `int`
- ▶ `double + double` will yield an `double`
- ▶ `String + String` will yield a `String`

UNARY ARITHMETIC OPERATORS

++ increments by 1 -- decrements by 1 Negation -expr

Post-increment *// analogous for decrement*

- ▶ If x takes 4, then $x++$ evaluates to 4 and afterwards x is incremented by 1

Pre-increment *// analogous for decrement*

- ▶ If x takes 4, then $++x$ evaluates to 5 since x is incremented by 1 beforehand

Given `int t = 3`, what is the value of t after $t = -t++ - ++t$?

$$\text{eval}(-t++ - ++t) = (-\text{eval}(t++)) - \text{eval}(++t) = (-3) - 5 = -3 - 5 = -8$$

RELATIONAL OPERATORS

Used to compare arithmetic expressions to make business logic decisions

$Aexpr_1 > Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is greater than $Aexpr_2$
$Aexpr_1 \geq Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is greater than or equal to $Aexpr_2$
$Aexpr_1 < Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is lower than $Aexpr_2$
$Aexpr_1 \leq Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is lower than or equal to $Aexpr_2$
$Aexpr_1 == Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is equal to $Aexpr_2$
$Aexpr_1 \neq Aexpr_2$	Evaluates to <i>true</i> if $Aexpr_1$ is different from $Aexpr_2$

They evaluate to **boolean**

LOGICAL OPERATORS

Used to compare boolean expressions to make business logic decisions

$!Bexpr$	Evaluates to <i>true</i> if <i>Bexpr</i> is <i>false</i>	
$Bexpr_1 \& Bexpr_2$	Evaluates to <i>true</i> if <i>Bexpr₁</i> is <i>true</i> and <i>Bexpr₂</i> is <i>true</i>	
$Bexpr_1 Bexpr_2$	Evaluates to <i>true</i> if <i>Bexpr₁</i> is <i>true</i> or <i>Bexpr₂</i> is <i>true</i>	
$Bexpr_1 \&\& Bexpr_2$	Evaluates to <i>true</i> if <i>Bexpr₁</i> is <i>true</i> and <i>Bexpr₂</i> is <i>true</i>	SC
$Bexpr_1 Bexpr_2$	Evaluates to <i>true</i> if <i>Bexpr₁</i> is <i>true</i> or <i>Bexpr₂</i> is <i>true</i>	SC

Short-Circuit evaluation

- ▶ In $Bexpr_1 \&\& Bexpr_2$, if *Bexpr₁* is *false* then *Bexpr₂* is not evaluated
- ▶ In $Bexpr_1 || Bexpr_2$, if *Bexpr₁* is *true* then *Bexpr₂* is not evaluated
- ▶ Efficient, but may lead to unintended behaviors if *Bexpr₂* has side-effects

ASSIGNMENT OPERATORS

Used to explicitly initialize a variable, or to change its value

<i>var = expr</i>	Evaluates <i>expr</i> and assigns the value to <i>var</i>
<i>var += expr</i>	Same as <i>var = var + expr</i>
<i>var -= expr</i>	Same as <i>var = var - expr</i>
<i>var *= expr</i>	Same as <i>var = var * expr</i>
<i>var /= expr</i>	Same as <i>var = var / expr</i>
<i>var %= expr</i>	Same as <i>var = var % expr</i>

For instance, given `int x = 4`

- ▶ `x -= 3` will assign `x` with 1
- ▶ `x += x - 7` will assign `x` with 1, equivalent to `x = x + (x - 7)`

JAVA CONTROL-FLOW CONSTRUCTS



CODE BLOCKS

List of instructions enclosed by braces { ... }

- ▶ Each block starts a new **scope** for variables
- ▶ Variables declared into a block are available into that block only

```
{ 1
    int j = 4; 2
    j++; // j takes 5 here 3
} 4
{ 5
    int j = 7; 6
    --j; // j takes 6 here 7
} 8
// j not available here 9
```

CODE BLOCKS (CONT'D)

A code block may exist on its own or it can belong to conditionals and loops

- ▶ A code block inherits the scope of the blocks to which it belongs

When code blocks contain a single instruction the braces can be omitted

`{ x = 2; }` is equivalent to `x = 2;`

CONDITIONAL STATEMENTS

Instructions allowing to dynamically select the code to execute

if (Bexpr) block and if (Bexpr) block else block

<code>if (i < 0) j++;</code>	1
<code>else { j--; }</code>	2
<code>// the value of j here depends on i</code>	3

 0 and `null` are not equivalent to *false*

CONDITIONAL STATEMENTS (CONT'D)

Case statements, they simplify if-else chains

```
switch (Aexpr) {  
    case Aexpr1: block1 break;  
    ...  
    case Aexprn: blockn break;  
    default: block break;  
}
```

```
switch (month) {  
    case 4:  
    case 6:  
    case 9:  
    case 11:  
        days = 30;  
        break;  
    case 2:  
        days = 28;  
        break;  
    default:  
        days = 31;  
        break;  
}
```

Only **byte**, **short**, **char**, **int** and **long** allowed (String and enum added from Java 7)

LOOP STATEMENTS

Instructions allowing to dynamically repeat code execution

`while (Bexpr) block` and `do (Bexpr) block while (Bexpr);`

```
int s = 0;           1
int i = 0;           2
while (i <= 9) {      3
    s += i;           4
    i++;              5
}                    6
// s takes 45 here    7
```

```
int s = 0;           1
int i = 0;           2
do {                 3
    s += i;           4
    i++;              5
} while (i <= 9);     6
// s takes 45 here    7
```

LOOP STATEMENTS (CONT'D)

Gauss at his 7

$$\sum_{k=0}^n k = \frac{n(n+1)}{2}; (k, n) \in \mathbb{N}^2$$



Me at my 19

$$77 + 33 = 100$$



CONTROLLED LOOPS

Simplified loops for controlled iterations

for (*init*; *Bexpr*; *inc/dec*) { *block* }

```
int s = 0;                                1
for (int i = 0; i <= 9; i++) {            2
    s += i;                                3
}                                           4
// s takes 45 here                          5
```

Simpler code

```
int s = 0;                                1
int i, j;                                  2
for (i = 0, j = 9; i <= 9 && j >= 0; i++, j--) 3
    s += i + j;                            4
// s takes 90 here                          5
```

Multiple init
Multiple inc/dec

DIRECT FLOW CONTROL STATEMENTS

Instructions that override the logical check in conditionals and loops

<code>break;</code>	Jump outside a loop (end the loop)
<code>continue;</code>	Jump to the next loop iteration
<code>return <i>expr</i>;</code>	Exit from a method and return the value of <i>expr</i>

```
int s = 0; 1
for (int i = 0; i <= 9; i++) { 2
    if (i == 4) break; 3
    s += i; 4
} 5
// s takes 6 instead of 45 6
```

```
int s = 0; 1
for (int i = 0; i <= 9; i++) { 2
    if (i == 4) continue; 3
    s += i; 4
} 5
// s takes 41 instead of 45 6
```

Q + A

